ESIGELEC

ÉCOLE D'INGÉNIEURS-ES GÉNÉRALISTES

Déploiement de solutions IA et cybersécurité

Cours n°3 : Scalabilité, Performance et Monitoring

Présenté par : Hichem AMMAR-BOUDJELAL

le : 10/11/2026

PROGRAMME DU COURS COMPLET

ESIGELEC
ÉCOLE D'INGÉNIEURS-ES GÉNÉRALISTES

1 - INTRODUCTION AU DÉPLOIEMENT D'IA ET À LA CYBERSÉCURITÉ



- CYCLE DE VIE D'UN PROJET IA
- CONCEPTS DE MLOPS ET CI/CD
- PREMIERS PRINCIPES DE SÉCURITÉ ET DE CONFORMITÉ
- TP N°1 : ÉTUDE DE CAS

2 - CONTENEURISATION ET INFRASTRUCTURE CLOUD



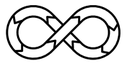
- DOCKER, IMAGES, CONTENEURS, REGISTRES
- KUBERNETES (GKE, KAPSULE, AKS, EKS)
- DÉPLOIEMENT D'UN SERVICE D'INFÉRENCE IA
- SCALABILITÉ HORIZONTALE ET VERTICALE

3 - SCALABILITÉ, PERFORMANCE ET MONITORING



- ARCHITECTURE SCALABLE ET TOLÉRANTE AUX PANNES
- AUTOSCALING, LOAD BALANCING ET TESTS DE CHARGE
- OBSERVABILITÉ : LOGS, MÉTRIQUES, TRACES
- MONITORING AVEC PROMETHEUS, GRAFANA, CLOUD MONITORING

4 - MLOPS ET PIPELINES D'INTÉGRATION CONTINUE



- CI/CD POUR MODÈLES IA
- VERSIONING DES MODÈLES ET JEUX DE DONNÉES
- GITHUB ACTIONS, CLOUD BUILD, VERTEX PIPELINES
- TESTS D'INTÉGRATION ET VALIDATION AUTOMATIQUE

5 - SÉCURITÉ DES API ET GESTION DES IDENTITÉS



- AUTHENTIFICATION (OAUTH2, JWT)
- AUTORISATION (IAM, RÔLES, CLÉS D'ACCÈS)
- CHIFFREMENT, SIGNATURES ET LOGS D'AUDIT
- VULNÉRABILITÉS TYPIQUES DES API IA

6 - ATTAQUES ET DÉFENSES IA



- ATTAQUES ADVERSARIALES ET DATA POISONING
- EXTRACTION ET INVERSION DE MODÈLES
- FUZZING ET INJECTION D'INFÉRENCES
- BLUE TEAM VS RED TEAM : STRATÉGIES DÉFENSIVES

7 - RGPD, IA RESPONSABLE ET ÉTHIQUE



- PRINCIPES RGPD : CONSENTEMENT, MINIMISATION, DROIT À L'OUBLI
- OUTILS D'ANONYMISATION ET DÉTECTION DE PII (NER)
- TRANSPARENCE, EXPLICABILITÉ ET TRAÇABILITÉ
- AUDITABILITÉ ET DOCUMENTATION IA

8 - GOUVERNANCE, DEVSECOPS ET AUDIT



- INTÉGRATION SÉCURITÉ + CONFORMITÉ DANS LE PIPELINE CI/CD
- RÔLES ET RESPONSABILITÉS (DEV, SEC, OPS, DPO, DATA)
- OUTILS DE SCAN (TRIVY, DEPENDABOT, SONARCLOUD)
- GOUVERNANCE DES MODÈLES, DÉRIVE ET SUPERVISION LONG TERME

1 - INTRODUCTION : DE LA ROBUSTESSE À LA RÉSILIENCE

EST-CE QU'ON VEUT JUSTE UN SYSTÈME SCALABLE ?

1 - INTRODUCTION : DE LA ROBUSTESSE À LA RÉSILIENCE

ON VEUT UN SYSTÈME RÉSILIENT

ESIGELEC
ÉCOLE D'INGÉNIEURS-ES GÉNÉRALISTES

1 - INTRODUCTION : DE LA ROBUSTESSE À LA RÉSILIENCE



EXEMPLE D'UN SYSTÈME RÉSILIENT

ESIGELEC
ÉCOLE D'INGÉNIEURS-ES GÉNÉRALISTES

1 - INTRODUCTION : DE LA ROBUSTESSE À LA RÉSILIENCE

UN SYSTÈME RÉSILIENT EST CAPABLE DE :

- **SURMONTER UNE DIFFICULTÉ OU UN TRAUMATISME**
- **SE RECONSTRUIRE**

1 - INTRODUCTION : DE LA ROBUSTESSE À LA RÉSILIENCE

DANS UN CADRE PUREMENT IT, UN SYSTÈME RÉSILIENT EST CAPABLE DE :

- **RÉSISTER AUX PANNES**
- **S'ADAPTER À LA CHARGE**
- **ÊTRE OBSERVABLE ET MESURABLE**

1 - INTRODUCTION : DE LA ROBUSTESSE À LA RÉSILIENCE



DEHORS C’EST LA GUERRE !

IL FAUT PRÉPARER VOTRE SYSTÈME À ENCAISSER LE PIRE

ESIGELEC
ÉCOLE D'INGÉNIEURS-ES GÉNÉRALISTES

2 - ARCHITECTURE SCALABLE

ESIGELEC
ÉCOLE D'INGÉNIEURS-ES GÉNÉRALISTES

SCALABILITÉ : CAPACITÉ D'UN SYSTÈME À AUGMENTER OU RÉDUIRE SES RESSOURCES SELON LA CHARGE

2 POSSIBILITÉS :

- **VERTICALE :** AJOUTER DES RESSOURCES (RAM/CPU/GPU)
- **HORIZONTALE :** AJOUTER PLUS DE NOEUDS/PODS

ÉLASTICITÉ : SCALABILITÉ AUTOMATIQUE ET DYNAMIQUE

- QUAND LE CLOUD S'ADAPTE SELON DES MÉTRIQUES (RÈGLES)
  - HORIZONTAL POD AUTOSCALER
  - VERTICAL POD AUTOSCALER

2 - ARCHITECTURE SCALABLE

ESIGELEC
ÉCOLE D'INGÉNIEURS-ES GÉNÉRALISTES

SIMULATION DE SCALABILITÉ HORIZONTALE

```
import time
import concurrent.futures

def infer(text):
    time.sleep(0.2)  # simulate inference latency
    return {"toxicity": 0.73}

def simulate_load(concurrency=1, n_requests=10):
    start = time.time()
    with concurrent.futures.ThreadPoolExecutor(max_workers=concurrency)
as ex:
        list(ex.map(infer, ["text"] * n_requests))
    duration = time.time() - start
    print(f"→ {n_requests} requêtes avec {concurrency} workers :
{duration:.2f}s")

simulate_load(concurrency=1, n_requests=10)
simulate_load(concurrency=5, n_requests=10)
simulate_load(concurrency=10, n_requests=10)
```

2 - ARCHITECTURE TOLÉRANTE AUX PANNES

TOLÉRANCE AUX PANNES : LE SYSTÈME CONTINUE DE FONCTIONNER MALGRÉ DES DÉFAILLANCES PARTIELLES

PRINCIPES :

- Réplication
  - Plusieurs pods
  - Plusieurs zones géographiques
- Redémarrage automatique → dû à un crash applicatif ou Out-of-Memory
- Stockage persistant et distribué
- **Pod Disruption Budget (PDB) :** permet de limiter le nombre d'interruptions volontaires simultanées des pods - **variable maxUnavailable**
- **Rolling Update :** Stratégie de mise à jour progressive d'une application où les anciennes instances sont remplacées une par une par des nouvelles

L'IDÉE N'EST PAS D'ÉVITER LES CRASHS, MAIS DE S'Y PRÉPARER

ESIGELEC
ÉCOLE D'INGÉNIEURS-ES GÉNÉRALISTES

2 - ARCHITECTURE TOLÉRANTE AUX PANNES

RÉPLICATION MULTI-PODS

```
spec:
   replicas: 3
```

3 PODS IDENTIQUES TOURNENT SIMULTANÉMENT.
KUBERNETES SURVEILLE ET REMPLACE LES PODS MANQUANTS.

RÉPLICATION MULTI-ZONES

```
gcloud container clusters create-auto ia-cluster --region=europe-west1
```

UN CLUSTER RÉGIONAL DISTRIBUE AUTOMATIQUEMENT LES PODS SUR PLUSIEURS ZONES

POSSIBILITÉ DE CRÉER DES CLUSTERS DANS PLUSIEURS RÉGIONS : GKE MULTI-CLUSTER INGRESS, MULTI-CLUSTER ORCHESTRATOR, ETC...

2 - ARCHITECTURE TOLÉRANTE AUX PANNES

REDÉMARRAGE AUTO

```
livenessProbe:
  httpGet:
    path: /health
    port: 8000
```

Si /health échoue plusieurs fois, Kubernetes tue et relance le conteneur

STOCKAGE PERSISTANT

```
volumes:
- name: cache
  persistentVolumeClaim:
    claimName: model-cache
```

Permet de conserver un cache de modèles même si un pod redémarre (Compute Engine Persistent Disk)

2 - ARCHITECTURE TOLÉRANTE AUX PANNES

HORIZONTAL POD AUTOSCALER

LE HPA SURVEILLE LA CHARGE CPU MOYENNE.

SI ELLE DÉPASSE 70%, IL CRÉE DE NOUVEAUX PODS.

SI ELLE REDESCEND, IL LES SUPPRIME.

LE CLUSTER AUTOSCALER VIENT EN COMPLÉMENT POUR LE PHYSIQUE.

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: ia-api-hpa }
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: ia-api
  minReplicas: 3
  maxReplicas: 15
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```

ESIGELEC
ÉCOLE D'INGÉNIEURS-ES GÉNÉRALISTES

2 - ARCHITECTURE TOLÉRANTE AUX PANNES

LOAD BALANCING

RÉPARTIT LE TRAFIC ENTRE PLUSIEURS PODS

GCP :

- **INTERNAL LOAD BALANCER**
  - **INTRA-VPC**
- **EXTERNAL LOAD BALANCER**
  - **ACCÈS PUBLIC**

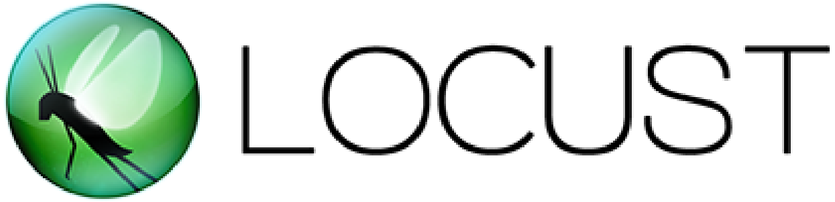
```
apiVersion: v1
kind: Service
metadata: { name: ia-api }
spec:
  selector: { app: ia-api }
  ports:
  - port: 80
    targetPort: 8080
  type: LoadBalancer
```

TYPE: LOADBALANCER → GKE CRÉE AUTOMATIQUEMENT UNE IP PUBLIQUE ET UN RÉPARTITEUR GLOBAL

IL VEILLE À CE QUE CHAQUE REQUÊTE SOIT DIRIGÉE VERS UN POD SAIN ET DISPONIBLE.

3 - TESTS DE CHARGE ET DE PERFORMANCE







MÉTRIQUES CLÉS :

- **LATENCE MOYENNE**
- **P95 / P99 :** Percentiles de latence
  - P95 → 95% des requêtes ont une latence inférieure à cette valeur
- **TAUX D'ERREUR**
- **DÉBIT (REQUÊTES/SECONDE)**

3 - TESTS DE CHARGE ET DE PERFORMANCE

- **ENVOI DE 50 REQUÊTES SIMULTANÉES**
  - Choisissez le message de votre choix ou une liste de messages

```
import asyncio, aiohttp, time

API_URL = "https://your-api-url.run.app/predict"

async def test(session, text):
    async with session.post(API_URL, json={"text": text}) as r:
        return await r.json()

async def load_test(n_users=50):
    start = time.time()
    async with aiohttp.ClientSession() as session:
        tasks = [test(session, "Hello world!") for _ in range(n_users)]
        await asyncio.gather(*tasks)
    print(f"{n_users} requêtes exécutées en {time.time()-start:.2f}s")

asyncio.run(load_test())
```

ESIGELEC
ÉCOLE D'INGÉNIEURS-ES GÉNÉRALISTES

3 - TESTS DE CHARGE ET DE PERFORMANCE

ENVIRONNEMENT DE TEST :

- **Plateforme : Google Kubernetes Engine (GKE)**
- **Version Kubernetes : 1.26**
- **Nombre de noeuds : 3 noeuds VM Compute Engine**
- **Type de machine : e2-standard-4 (4 vCPU, 16 Go RAM)**
- **Application testée : Application web déployée dans des pods Kubernetes**
- **Outils de tests de charge : k6, Locust, JMeter, etc...**

MÉTRIQUES MESURÉES :

- **Débit de requêtes / seconde**
- **Taux de latence (moyenne, P95, P99, maximum)**
- **Taux d'erreur HTTP (5XX, 4XX)**
- **Utilisation CPU et mémoire des noeuds et pods**
- **Nombre de connexions simultanées**
- **Taux de mise à l'échelle automatique (scaling)**

3 - TESTS DE CHARGE ET DE PERFORMANCE

Scénario	Débit max (RPS)	Latence moyenne (ms)	Taux d'erreur (%)	CPU moyen (%)	Mémoire moyenne (%)	Observations
Montée en charge progressive	950	120	0,5	70	65	Autoscaling efficace jusqu'à 900 RPS
Charge soutenue 30 min	500	100	0	55	60	Stabilité du cluster confirmée
Montée rapide autoscaling	1000	140	2	85	72	Légère saturation CPU et erreurs HTTP 5xx

4 - MONITORING ET OBSERVABILITÉ

ESIGELEC
ÉCOLE D'INGÉNIEURS-ES GÉNÉRALISTES

3 PILIERS PRINCIPAUX

- **Logs :** Évènements horodatés
- **Métriques :** Valeurs agrégées
- **Traces :** Suivi des requêtes



CLOUD TRACE



PROMETHEUS



CLOUD LOGGING

4 - MONITORING ET OBSERVABILITÉ



PROMETHEUS

MÉTHODE N°1 :

- **MIDDLEWARE PRÊT À L'EMPLOI**

```
from prometheus_fastapi_instrumentator import Instrumentator

app = FastAPI()
Instrumentator().instrument(app).expose(app, endpoint="/metrics")
```

EXPOSE LA LATENCE, LES REQUÊTES/SECONDE, ERREURS HTTP, ETC...

4 - MONITORING ET OBSERVABILITÉ

ESIGELEC
ÉCOLE D'INGÉNIEURS-ES GÉNÉRALISTES



PROMETHEUS

MÉTHODE N°2 :

- **INSTRUMENTATION MANUELLE**

```
from prometheus_client import Histogram
LATENCY = Histogram("inference_latency_seconds", "Latence d'inférence")
@app.post("/predict")
def predict(...):
  with LATENCY.time():
    return run_inference(...)
```

PERMET DE MESURER PRÉCISÉMENT L'INFÉRENCE IA

4 - MONITORING ET OBSERVABILITÉ

ESIGELEC
ÉCOLE D'INGÉNIEURS-ES GÉNÉRALISTES

- PROMETHEUS COLLECTE
- GRAFANA VISUALISE
- GCP PERMET D'INTÉGRER LES DEUX VIA **MANAGED SERVICE FOR PROMETHEUS**
- DASHBOARD TYPIQUE :
  - REQUÊTES/SECONDE
  - LATENCE P95
  - TAUX D'ERREUR
  - CPU/MÉMOIRE PODS

PERMET DE MESURER PRÉCISÉMENT L'INFÉRENCE IA

4 - MONITORING ET OBSERVABILITÉ

ESIGELEC
ÉCOLE D'INGÉNIEURS-ES GÉNÉRALISTES

Grafana



5 - GESTION DES ALERTES

ESIGELEC
ÉCOLE D'INGÉNIEURS-ES GÉNÉRALISTES

POUR SURVEILLER DE MANIÈRE PROACTIVE LA SANTÉ DES SERVICES :

- ALERTES **SLO (SERVICE LEVEL OBJECTIVES)**

EXEMPLES :

- LATENCE > 500 MS → ALERTE
- CPU > 90% → ALERTE
- ERREUR 5XX > 1% → ALERTE

PERMET DE MESURER PRÉCISÉMENT L'INFÉRENCE IA

5 - GESTION DES ALERTES

ESIGELEC
ÉCOLE D'INGÉNIEURS-ES GÉNÉRALISTES



PROMETHEUS

- **DÉFINI DANS LE YAML**

OU DANS L'INTERFACE GRAFANA

```
groups:
- name: slo-latency
  rules:
  - record: slo:latency_under_300ms_ratio
    expr: |
      rate(api_latency_seconds_bucket{le="0.3"}[5m])
      /
      rate(api_requests_total[5m])
  - alert: LatencySLOViolated
    expr: slo:latency_under_300ms_ratio < 0.95
    for: 10m
    labels:
      severity: warning
    annotations:
      summary: "SLO latence violé (moins de 95% < 300ms)"
```

COMPARAISON DU NOMBRE DE REQUÊTES DONT LA LATENCE EST < 300 MS AU NOMBRE TOTAL DE REQUÊTES SUR UNE FENÊTRE DE 5 MINUTES.
SI LA PROPORTION EST < 95% PENDANT 10 MINUTES, L'ALERTE SE DÉCLENCHE.

ESIGELEC
ÉCOLE D'INGÉNIEURS-ES GÉNÉRALISTES

MERCI DE VOTRE ATTENTION



Contact :
Hichem AMMAR-BOUDJELAL
Mail : hichem@dpliance.com